

Aprendizado de Máquina B Otimização

Lourenço R. G. Araújo - DEST
Pedro O.S. Vaz de Melo - DCC

UFMG

Março de 2026

Otimização

- Sabemos que uma etapa crucial do treinamento de redes neurais consiste em encontrar um bom conjunto de parâmetros que leve ao bom desempenho a rede
- Gostaríamos de um conjunto ótimo - minimização de uma função objetivo por gradiente descendente
- Nosso objetivo real: obter um modelo que minimize o erro em um conjunto de teste desconhecido durante o treinamento - geralmente é um problema intratável
- Adotamos uma abordagem indireta de otimizar alguma medida de performance no conjunto de treinamento

Otimização

- Minimizar o erro de generalização corresponderia a minimizar a seguinte função objetivo:

$$J^*(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{data}} \mathcal{L}(\hat{y}, y)$$

- Para isso, precisaríamos conhecer a distribuição $p_{data}(\mathbf{x}, y)$
- Como não conhecemos, estamos presos ao conjunto de treinamento - um número limitado de pares de amostra $(\mathbf{x}, y)$
- A alternativa que adotamos segue o princípio de Minimização do Risco Empírico:

$$J(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{data}} \mathcal{L}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Otimização

- Minimizamos o risco empírico na esperança (baseada em resultados teóricos) de que o risco real esteja caindo
- Na prática, minimizar o risco empírico pode levar (e leva) a Overfitting
- Além das estratégias de regularização, algumas considerações precisam ser feitas a respeito da otimização

Otimização

- As condições de parada para o processo de treinamento de uma rede neural, com frequência, envolvem um conjunto de validação (olhar para o conjunto de validação seria o equivalente a avaliar o risco real)
- Por exemplo, é comum, durante o treinamento de redes neurais, interromper a otimização antes que as derivadas da função objetivo fiquem iguais a zero

Otimização

- Pensando na maximização da verossimilhança, resolver

$$\theta^* = \operatorname{argmax}_{\theta} \sum_{i=1}^m \log p_{\text{modelo}}(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

é equivalente a resolver:

$$\theta^* = \operatorname{argmax}_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} \log p_{\text{modelo}}(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

- De forma similar, o gradiente também pode ser visto como um valor esperado sobre a distribuição empírica:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} \nabla_{\theta} \log p_{\text{modelo}}(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

Otimização

- Sabemos que a média amostral é um estimador não viesado da média
- O desvio padrão de uma média estimada a partir de n amostras é $\sigma/\sqrt{n}$
- O termo $\sqrt{n}$ no denominador indica que não há ganhos lineares de variância quando são utilizadas muitas amostras para estimação do gradiente
 - Se usamos 10000 amostras ao invés de 100, gastamos 100 vezes mais computação para um ganho de precisão (queda na variância) de apenas 10 vezes
- Alternativa: usar minibatches para estimar nosso gradiente

Otimização

Para o treinamento de redes neurais, existem alguns desafios com os quais é necessário lidar:

- 1 Mínimos locais: para funções não-convexas, existem mínimos locais, para os quais o valor da função de custo pode ser grande, comparado ao valor do mínimo global (em geral, surpreendentemente, é um problema menor no treinamento de redes neurais)
- 2 Pontos de sela: especialmente em situações em que se usa derivadas de segunda ordem, pontos de sela são problemáticos. Gradiente descendente parece escapar de pontos de sela, embora o gradiente se aproxime de zero
- 3 Platôs: regiões com gradiente próximo de zero

Otimização

- 4 Regiões íngremes: regiões com valores grandes demais de gradiente
- 5 *Vanishing e exploding gradients*: acúmulo por multiplicação de valores de gradientes menores ou maiores que 1 (ocorre principalmente em redes recorrentes)

Gradiente Descendente Estocástico - SGD

- Principal algoritmo (com variações) para treinamento de redes neurais (especialmente redes neurais profundas)
- É possível obter uma estimativa não viesada do gradiente calculando o gradiente médio para um *minibatch* de amostras de treinamento
- Comparado ao algoritmo de gradiente descendente, o SGD demanda uma taxa de aprendizado decrescente ao longo do tempo
 - A amostragem de um minibatch introduz um ruído que não desaparece, mesmo quando o gradiente tende a zero

Gradiente Descendente Estocástico - SGD

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

- Em geral, aplica-se decaimento linear à taxa de aprendizado, até que se atinge um valor mínimo

Momento

- A ideia de usar Momento surge com o objetivo de acelerar a convergência do algoritmo de gradiente descendente
- O algoritmo de GD pode ziguezaguear em alguns casos, o que atrasa a convergência
- Adiciona-se uma média móvel com decaimento exponencial ao negativo do gradiente, o que reduz o impacto de grandes mudanças de direção

Momento

- Introduzimos o conceito de velocidade:

$$\mathbf{v}^{(t)} = \alpha \mathbf{v}^{(t-1)} - \epsilon \nabla_{\theta}^{(t)} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$$

$$\theta^{(t+1)} = \theta^{(t)} + \mathbf{v}^{(t)}$$

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity $\mathbf{v}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

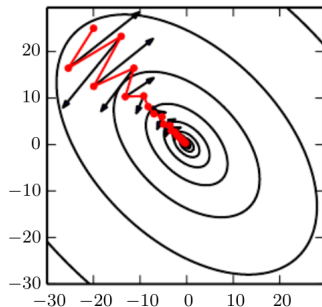
 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$.

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.

end while

Momento

- Existe ainda, o Momento de Nesterov, em que o gradiente é avaliado levando-se em conta uma correção que aplica a velocidade atual antes do cálculo do gradiente
- Considera-se uma atualização temporária: $\tilde{\theta}^{(t)} = \theta^{(t)} + \alpha \mathcal{V}^{(t)}$
- Computa-se o gradiente considerando-se os valores $\tilde{\theta}^{(t)}$



Taxas de aprendizado adaptativas

- A taxa de aprendizado é um parâmetro especialmente sensível de se ajustar
- Adaptamos as taxas de aprendizado para cada um dos parâmetros que desejamos otimizar

Taxas de aprendizado adaptativas

ADAGRAD

- Aplica-se um dimensionamento aos gradientes que é proporcional à soma do histórico dos gradientes ao quadrado
- Em direções com maiores gradientes, a taxa de aprendizado cai mais rapidamente
- Funciona muito bem em casos convexos

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$.

 Compute update: $\Delta \theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

Taxas de aprendizado adaptativas

RMSPROP

- Aprimora o algoritmo de ADAGRAD para diminuir o efeito do gradiente acumulado conforme mais iterações vão ocorrendo
- Ao invés do histórico completo do gradiente ao quadrado, utiliza-se uma média móvel com decaimento exponencial
- Com isso, acelera-se a convergência em casos de otimização não convexa
- É comum, combinar o RMSPROP com alguma estratégia de momento

Taxas de aprendizado adaptativas

RMSPROP

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers

Initialize accumulation variables $\mathbf{r} = 0$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$.

 Compute parameter update: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta \theta$.

end while

Taxas de aprendizado adaptativas

ADAM

- O algoritmo ADAM pode ser entendido como uma variante da combinação de RMSPROP com momento
- Tanto o gradiente quanto o gradiente ao quadrado são corrigidos com base em séries exponencialmente suavizadas
- Existe também uma correção de viés para as primeiras iterações de treinamento
- Geralmente, o ADAM é considerado robusto à escolha de hiperparâmetros

Taxas de aprendizado adaptativas

ADAM

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

Outras considerações

- Heurísticas de inicialização dos pesos
- Normalização de entradas
- Batch normalization